

Flowchart code used for the architecture diagram

A[Mobile Apps] --> R

B[Web Apps] --> R

C[Partner APIs] --> R

D[Internal Portals] --> R

R[RDAS REST API]

R --> V[Validation + AuthN/AuthZ]

R --> S[Service Layer]

S --> C1[Cache Layer]

S --> A1[Aggregation/Normalization Engine]

A1 --> W[SOAP Client Adapter]

W --> X[CountryInfo SOAP Service]

S --> O[Audit Logs / Metrics / Tracing]

S --> H[Health / Actuator Endpoints]

## **Responsibilities by layer**

### **API layer**

- Exposes REST endpoints.
- Validates inputs.
- Handles pagination, sorting, and filtering parameters.

### **Service layer**

- Applies business rules.
- Orchestrates cache lookup and SOAP fallback.

- Assembles results into a single response format.

### **SOAP adapter**

- Encapsulates all SOAP communication.
- Prevents SOAP details from leaking into the rest of the application.
- Makes the SOAP provider replaceable later.

### **Cache layer**

- Stores reference data and precomputed mappings.
- Reduces SOAP requests.
- Supports stale data during outages.

### **Observability**

- Logs every request with correlation ID.
- Metrics for cache hit ratio, SOAP latency, and failure rate.
- Health checks and alerting.

### **Why this architecture is strong**

It solves the exact business pain:

- One source of truth for all channels.
- One security boundary for SOAP credentials.
- Better performance through caching.
- Better user experience through pagination and filtering.
- Better resilience through stale-cache fallback.
- Better auditing because all requests pass through one service.